PIZZERÍA OIA

(Onai -Isaac-Alex)

MEMORIA DEL PROYECTO

MÉTODO 1

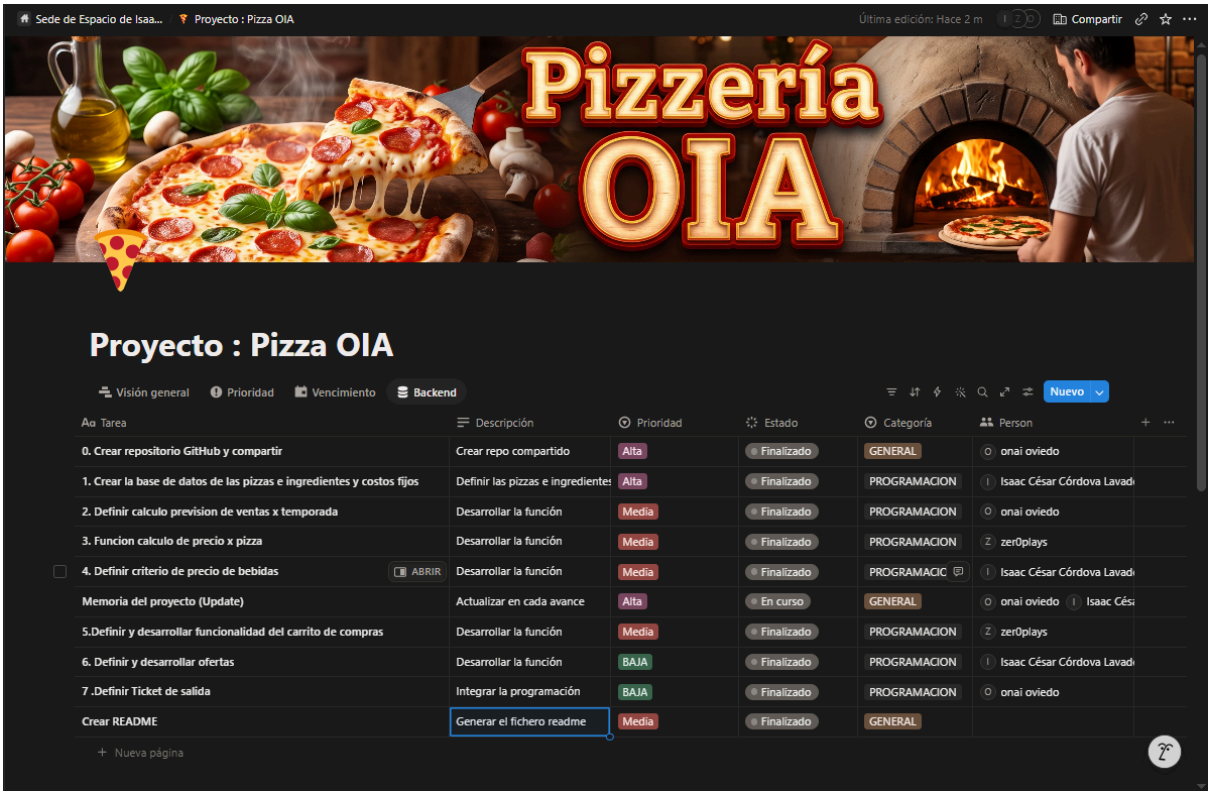
Avance : 07/10/26

- Selección de la aplicación de planificación y distribución de tareas.
- Creación de GitHub.
- Creación de la memoria.

1. Planificación inicial y distribución de tareas:

Tarea	Descripción	Prioridad	Estado	Categoría	Responsable(s)
0. Crear repositorio GitHub y compartir	Crear repo compartido	Alta	Backlog	GENERAL	@onai oviedo
1. Crear la base de datos de las pizzas e ingredientes y costos fijos	—	Alta	Backlog	PROGRAMACION	@Isaac César Córdova Lavado
2. Definir calculo prevision de ventas x temporada	—	Media	Backlog	PROGRAMACION	@onai oviedo
3. Funcion calculo de precio x pizza	—	Media	Backlog	PROGRAMACION	@zer0plays
4. Definir criterio de precio de bebidas	—	Media	Backlog	PROGRAMACION	@Isaac César Córdova Lavado
Memoria del proyecto (Update)	—	Alta	Backlog	GENERAL	@onai oviedo, @Isaac César Córdova Lavado
5.Definir y desarrollar funcionalidad del carrito de compras	—	Media	Backlog	PROGRAMACION	@zer0plays
6. Definir y desarrollar ofertas	—	BAJA	Backlog	PROGRAMACION	@Isaac César Córdova Lavado
7. Definir Ticket de salida	—	BAJA	Backlog	PROGRAMACION	@onai oviedo
Crear README	Generar el fichero readme	Media	Backlog	GENERAL	—

2. Herramienta de planificación : Notion



Enlace de Github: https://github.com/onaioviedo/Pizzeria-OIA
Enlace a Notion: 🍕 [Proyecto : Pizza OIA]

3. Metodología de Control de Versiones y Flujo de Trabajo en GitHub

Para garantizar la integridad del código fuente, minimizar la aparición de conflictos y permitir un desarrollo colaborativo eficiente entre los tres integrantes del equipo, se ha establecido un flujo de trabajo centralizado en GitHub bajo el repositorio Pizzeria - OIA

Estructura de Ramas (*Branching Model*)

Rama	Responsable / Rol	Propósito
main	Todo el equipo	Rama principal de producción con versiones estables y probadas del software.
Onai	Onai	Rama de trabajo individual para el desarrollo y prueba de sus módulos asignados.
Alex	Alex	Rama de trabajo individual para el desarrollo y prueba de sus módulos asignados.

feature-cesar	Isaac	Rama de características y desarrollo asignada a la implementación de funcionalidades.

4. Organización y Arquitectura del Repositorio

El código del proyecto se organiza de forma modular para evitar colisiones y mantener una clara separación de responsabilidades:

Pizzeria-OIA/
├── main/ # Punto de entrada, orquestación y lógica principal de la aplicación
├── functions/ # Módulos auxiliares, utilidades y funciones reutilizables de negocio
└── data/ # Ingredientes, Bebidas, Pizzas, Temporadas, Ofertas

5. Definición de la Base de datos en: Ingredientes, Pizzas, Ofertas y Temporadas.

Ingredientes:

- "harina": 1.20, # Harina de fuerza / trigo para pizza
- "tomate": 1.60, # Salsa de tomate triturado / passata
- "queso": 7.50, # Queso rallado mezcla / gouda
- "mozzarella": 8.20, # Mozzarella rallada / fior di latte
- "jamon": 7.90, # Jamón cocido / york en dados o lonchas
- "bacon": 8.50, # Bacon ahumado en tiras[cite: 1]
- "pollo": 6.80, # Pechuga de pollo cocinada / marinada[cite: 1]
- "atun": 9.50, # Atún claro en conserva escurrido[cite: 1]
- "champinones": 3.40, # Champiñón fresco laminado[cite: 1]
- "cebolla": 1.30, # Cebolla morada o dulce en juliana[cite: 1]
- "aceitunas": 4.50, # Aceitunas negras / verdes deshuesadas[cite: 1]
- "pepperoni": 11.20, # Pepperoni / salami especiado en rodajas[cite: 1]
- "pimenton": 4.20, # Pimiento rojo fresco en tiras[cite: 1]

Ingredientes añadidos para complementar la carta

- "carne_picada": 7.40, # Carne picada de vacuno/cerdo salteada
- "pina": 2.10, # Piña troceada escurrida
- "anchoas": 18.50, # Filetes de anchoa en aceite
- "gorgonzola": 12.80, # Queso azul / gorgonzola
- "parmesano": 15.60, # Queso parmesano curado rallado / lascas
- "salsa_barbacoa": 3.80, # Salsa barbacoa para pizza
- "jalapenos": 5.20, # Jalapeños encurtidos en rodajas
- "rucula": 9.80, # Rúcula fresca seleccionada
- "albahaca": 14.00, # Albahaca fresca
- "oregano": 12.00 # Orégano seco molido

Tipos de pizza

- # 1. Clásica básica
"Margarita": ["harina", "tomate", "mozzarella", "albahaca"],
- # 2. Clásica con jamón
"Prosciutto": ["harina", "tomate", "queso", "jamon", "oregano"],
- # 3. La tradicional barbacoa
"Barbacoa": ["harina", "tomate", "queso", "bacon", "pollo", "carne_picada", "salsa_barbacoa"],
- # 4. Clásica con pepperoni
"Pepperoni": ["harina", "tomate", "mozzarella", "pepperoni", "oregano"],
- # 5. La clásica combinación dulce y salada
"Hawaiana": ["harina", "tomate", "mozzarella", "jamon", "pina"],
- # 6. Variedad con champiñón y jamón
"Capricciosa": ["harina", "tomate", "mozzarella", "jamon", "champinones", "aceitunas"],
- # 7. Selección de quesos
"Cuatro_Quesos": ["harina", "tomate", "mozzarella", "queso", "gorgonzola", "parmesano"],
- # 8. Sabor marinero
"Marinara_Especial": ["harina", "tomate", "mozzarella", "atun", "cebolla", "aceitunas"],
- # 9. Con anchoas tradicional
"Napolitana": ["harina", "tomate", "mozzarella", "anchoas", "aceitunas", "oregano"],
- # 10. Pizza de huerto
"Vegetal": ["harina", "tomate", "mozzarella", "champinones", "cebolla", "pimenton", "aceitunas"],
- # 11. Con pollo y verdura
"Pollo_Grill": ["harina", "tomate", "queso", "pollo", "pimenton", "cebolla"],
- # 12. Estilo tex-mex picante
"Diabla": ["harina", "tomate", "mozzarella", "pepperoni", "carne_picada", "jalapenos"],
- # 13. Toque crujiente y ahumado
"Carbonara_Pizza": ["harina", "queso", "mozzarella", "bacon", "cebolla", "champinones"],
- # 14. Pollo barbacoa dulce
"Pollo_BBQ": ["harina", "mozzarella", "pollo", "bacon", "cebolla", "salsa_barbacoa"],
- # 15. Pizza con carnes y toque fresco
"Boloñesa_Rústica": ["harina", "tomate", "queso", "carne_picada", "parmesano", "oregano"],
- # 16. Pizza fresca y gourmet
"Rúcula_y_Parmesano": ["harina", "tomate", "mozzarella", "rucula", "parmesano"],
- # 17. Fuerte y con carácter
"Gorgonzola_y_Bacon": ["harina", "mozzarella", "gorgonzola", "bacon", "cebolla"],
- # 18. Fusión atún y bacon
"Atún_Especial": ["harina", "tomate", "queso", "atun", "bacon", "pimenton", "aceitunas"],
- # 19. Explosiva con jalapeño
"Mexicana_Pollo": ["harina", "tomate", "queso", "pollo", "jalapenos", "pimenton", "cebolla"],
- # 20. Gran carnívora
"Carnívora_Suprema": ["harina", "tomate", "mozzarella", "jamon", "bacon", "pepperoni", "carne_picada"]

Ofertas:

- 2x1 en pizzas concretas ["Margarita", "Pepperoni", "Hawaiana"]
- Menú pizza + bebida.
- Descuento del 10% si el pedido supera 30 euros.

Temporadas

```
#Funcion Temporada
temporada = "media"
match temporada:
    case "baja":
        pizzas_estimadas_mes = 1000
    case "media":
        pizzas_estimadas_mes = 1850
    case "alta":
        pizzas_estimadas_mes = 3000
    case _:
        pizzas_estimadas_mes = 1000
```

Avance : 14/10/26

- Definicion del protocolo de subidas a Github
- Issue 2. Definir calculo prevision de ventas x temporada COMPLETADA (Onai)
- Issue 3. Funcion calculo de precio x pizza COMPLETADA (Alex)
- Issue 4. Definir criterio de precio de bebidas COMPLETADA (Isaac)
- Issue 5.Definir y desarrollar funcionalidad del carrito de compras EN CURSO (Alex)
- Issue 6. Definir y desarrollar ofertas EN CURSO (Isaac)

1. Protocolo de Sincronización y Subida (*Workflow*)

Para trabajar en equipo sin pisar código ni generar errores en producción, se definió un ciclo de 4 pasos obligatorios antes de cada entrega**:**

[1. Actualizar] ⟶ [2. Integrar en local] ⟶ [3. Avisar al equipo] ⟶ [4. Subida final]

- Actualizar el entorno local:

git checkout main
git pull origin main

- **Integrar y resolver conflictos en local:**

Fusionar la versión actualizada de `main` en la rama personal (`Onai`, `Alex` o `feature-cesar`), resolver colisiones en `functions/` o `data/` y comprobar que el proyecto funciona:

git checkout <mi-rama>
git merge main

- **Aviso de bloqueo al equipo:**

Notificar por el canal interno (*"Voy a subir cambios a main"*) para congelar subidas simultáneas del resto de compañeros.

- **Publicación definitiva en GitHub:**

git checkout main
git merge <mi-rama>
git push origin main

Avance : 21/10/26

- Issue 5.Definir y desarrollar funcionalidad del carrito de compras COMPLETADA (Alex)
- Issue 6. Definir y desarrollar ofertas COMPLETADA (Isaac)
- Issue 7. Definir Ticket de salida COMPLETADA (Onai)
- Fase de pruebas y depuracion final QA (Todos)
- README.md COMPLETADO

1. Fase de Pruebas y Depuración Final (*QA*)

Se decidió realizar una batería de pruebas de integración para validar el sistema antes de la entrega definitiva. Esta fase resultó clave para detectar y corregir errores críticos en la lógica de negocio:

Motor de ofertas (functions/): Se detectaron fallos al aplicar promociones (descuentos que no se computaban, condiciones erróneas en carritos mixtos o duplicidades en ofertas especiales). Se solventó reestructurando las condiciones lógicas de activación.

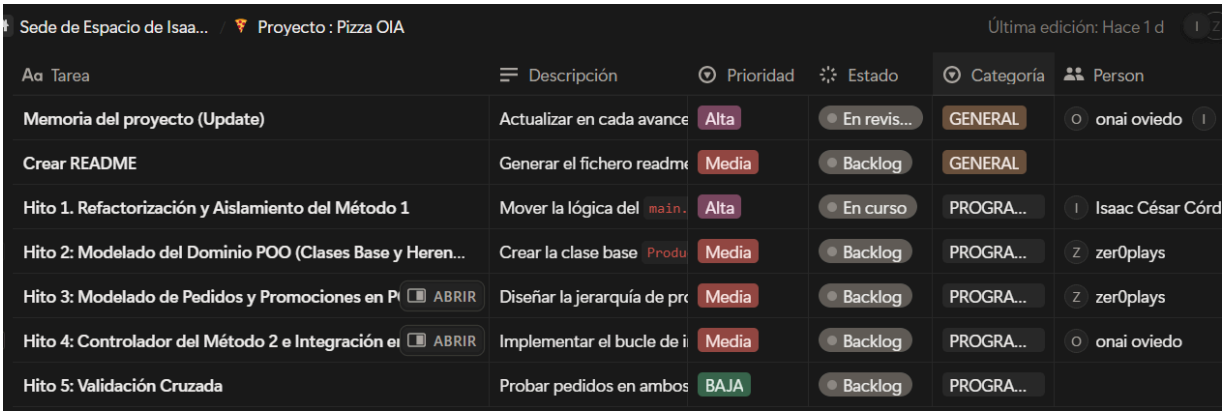
Cálculo de precios (functions/ y data/): Aparecieron descuadres entre el precio base, suplementos por ingredientes y redondeos del total. Se normalizó la fórmula para fijar un orden estricto: subtotal → aplicación de promociones → total final.

Consistencia de datos (data/): Se corrigieron errores al guardar tickets con promociones activas, asegurando que los datos registrados coincidieran exactamente con el cobro mostrado.

MÉTODO 2 : Implementación nueva:

- Añadir una nueva programación del código POO y clases.

Nueva planificación:



Estructura de carpetas del proyecto:

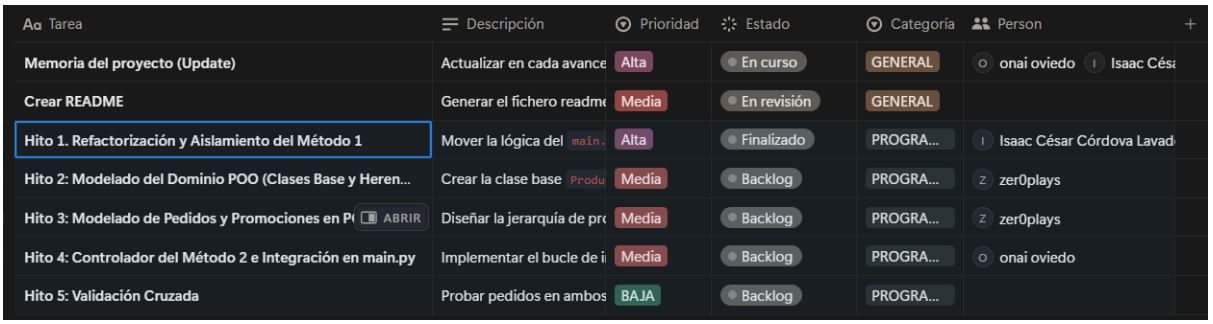
Pizzeria-OIA/

```
├─ data.py                    # Catálogo centralizado compartido (ingredientes, pizzas, bebidas, constantes)
├── metodo_procedural.py      # MÉTODO 1: Código original aislado en una función ejecutar_metodo_1()
├── metodo_oop.py             # MÉTODO 2: Programación Orientada a Objetos (POO)
│                             #   - Clases de productos (Producto, Pizza, Bebida) con herencia y super()
│                             #   - Clases de promociones (Promocion, Promocion2x1, etc.)
│                             #   - Clase Pedido / Carrito
│                             #   - Función ejecutar_metodo_2()
└── main.py                   # Punto de entrada único con menú para elegir Método 1 o Método 2
```

Avance: 02/11/26

- Renombrado de archivos function.py a “metodo_procedural.py”, para aislar el metodo 1 y creacion del archivo “metodo_opp.py”, para desarrollar el metodo 2.
- Limpieza del main para solo invocar los 2 métodos al ejecutar el código.

Estado del tablero:



Tarea	Descripción	Prioridad	Estado	Categoría	Person
Memoria del proyecto (Update)	Actualizar en cada avance	Alta	En curso	GENERAL	onai oviedo, Isaac Cés
Crear README	Generar el fichero readm	Media	En revisión	GENERAL	
Hito 1. Refactorización y Aislamiento del Método 1	Mover la lógica del main.	Alta	Finalizado	PROGRA...	Isaac César Córdova Lavad
Hito 2: Modelado del Dominio POO (Clases Base y Heren...	Crear la clase base Produ	Media	Backlog	PROGRA...	zer0plays
Hito 3: Modelado de Pedidos y Promociones en P ABRIR	Diseñar la jerarquía de pr	Media	Backlog	PROGRA...	zer0plays
Hito 4: Controlador del Método 2 e Integración en main.py	Implementar el bucle de i	Media	Backlog	PROGRA...	onai oviedo
Hito 5: Validación Cruzada	Probar pedidos en ambos	BAJA	Backlog	PROGRA...	

Ramas del Método 2

Para el Método 2 se siguió el mismo protocolo de subida, pero se creó **una rama feature por cada bloque de clases**. Así cada bloque se podía integrar en `main` por separado:

Rama	Contenido
`feature/poo-productos`	Clases `Producto`, `Pizza`, `Bebida` y `Carta`
`feature/poo-carrito`	Clases `Comensal` y `Carrito`, y el ticket
`feature/poo-ofertas`	Clases `Oferta`, `Oferta2x1`, `OfertaMenu`, `OfertaVolumen` y `GestorOfertas`
`feature/poo-pizzeria`	Clase `Pizzeria`, función `ejecutar()` y actualización del README

Estructura del repositorio

```
None
Pizzeria-OIA/
├── main.py                 # Punto de entrada
├── data/
│   └── data.py             # Catálogo compartido por los dos métodos
└── src/
    ├── functions.py        # MÉTODO 1: funciones procedurales
    └── clases/             # MÉTODO 2: POO
        ├── productos.py    # Producto, Pizza y Bebida (herencia y
super())
        ├── carta.py        # Carta: crea y busca pizzas y bebidas
        ├── carrito.py      # Comensal y Carrito (subtotal y ticket)
        ├── ofertas.py      # Oferta, Oferta2x1, OfertaMenu,
OfertaVolumen y GestorOfertas
        └── pizzeria.py     # Pizzeria: temporada, costes y pedido;
ejecutar()
```

Cambios en el repositorio

Respecto a la estructura del Método 1 (`main/`, `functions/`, `data/`), el repositorio se reorganizó así:

1. **Separación de datos y código.** El catálogo (ingredientes, pizzas, bebidas y constantes) está ahora en un único archivo, `data/data.py`. Los dos métodos lo importan, así que solo hay una fuente de datos y cualquier cambio de precio afecta a los dos por igual.
2. **Todo el código fuente está en `src/`.** El Método 1 queda en `src/functions.py`, sin cambios en su lógica. Así se conserva la versión procedural como referencia.
3. **Nueva carpeta `src/clases/` para el Método 2.** Cada responsabilidad tiene su propio módulo (productos, carta, carrito, ofertas y pizzería), en lugar de tenerlo todo en un único archivo. Cada módulo corresponde a una de las ramas feature, por lo que se pudieron integrar en `main` sin conflictos.
4. **`main.py` solo sirve como punto de entrada.** Se eliminó la lógica de negocio y solo se encarga de lanzar los métodos. Falta el menú de selección (tarea 13).

Conceptos de POO aplicados

- **Herencia y `super()`:** `Pizza` y `Bebida` heredan de `Producto`, y las tres ofertas heredan de `Oferta`.
- **Polimorfismo:** cada oferta tiene sus propios `calcular_descuento()` y `descripcion()`, y `GestorOfertas` las aplica todas de la misma forma, sin saber de qué tipo es cada una.
- **Encapsulación:** cada `Pizza` calcula su coste y su precio, y el `Carrito` genera su propio ticket.
- **Reutilización:** el Método 2 usa los mismos datos de `data/data.py` que el Método 1.

Avance : 09/11/2026

- Añadir un sistema de autenticación mediante correo y contraseña.
- Actualizar el readme y la memoria para subirlo en Github

Módulo de Inicio de Sesión (Login)

Para asegurar el acceso a la caja registradora de la pizzería y mantener todo el código ordenado, hemos decidido aislar la parte del login en su propio archivo (login.py) y conectarlo con el núcleo principal (main.py). Así evitamos recargar el archivo principal y mantenemos una estructura limpia.

login.py

1. ¿Cómo funciona la validación de los datos?

El sistema comprueba que los datos introducidos cumplan con los filtros de seguridad antes de dejar pasar a nadie:

- **El correo electrónico:** Usamos expresiones regulares (re) para asegurarnos de que lo que se escribe tenga pinta de correo real (que lleve texto, una arroba @, un dominio y su extensión, como .com o .es).
- **La contraseña:** Le hemos puesto unas reglas estrictas para que sea segura. Como mínimo tiene que tener 8 caracteres, incluir obligatoriamente al menos una minúscula, una mayúscula y un símbolo o carácter especial (!, @, #, etc.).

2. Pensando en la comodidad de uso (Experiencia de usuario)

Queríamos que usar la consola no fuera frustrante si te equivocas al teclear, por lo que le hemos añadido detalles prácticos:

- **Reintentos automáticos:** Si metes mal el correo, la contraseña o te equivocas de clave, el programa no se cierra de golpe. Te avisa con un mensaje claro y te vuelve a pedir los datos desde el principio.
- **Guías visuales:** Antes de pedirte la contraseña, la consola te recuerda brevemente qué formato necesita para que no tengas que adivinarlo.
- **Opción de salida:** Si en algún momento quieres cancelar la prueba o salir del sistema sin tener que completar el login, basta con escribir exit en cualquiera de las pantallas para que el programa se cierre de forma ordenada.